

## EXPERIMENT NO.3

**Aim:** To study Deployment and publish smart contracts on Ethereum test network.

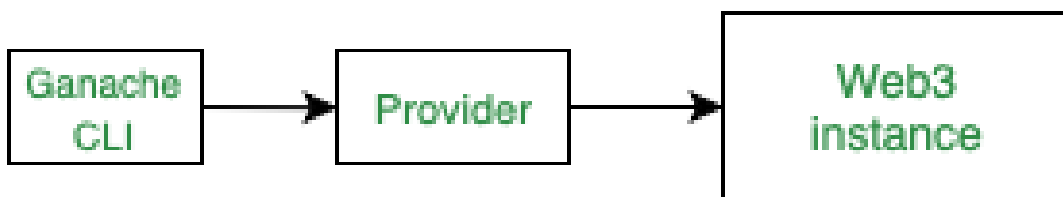
### Theory:

#### Smart Contract:

Smart contract are blocks of code that reside on the blockchain. It is like an Ethereum account but there is a critical difference between an external account and a smart contract. Unlike a smart contract, an external account can connect to multiple Ethereum networks (Goerli testnet, mainnet, etc.) whereas a smart contract is only specific to one individual network (the network it is deployed on). When a smart contract is deployed, it creates an instance (contract account) on the network. One can create multiple instances of a smart contract on the network or multiple networks. Deployment of a smart contract is done by sending a transaction to the network with bytecode.

#### Deploying To a Local Network

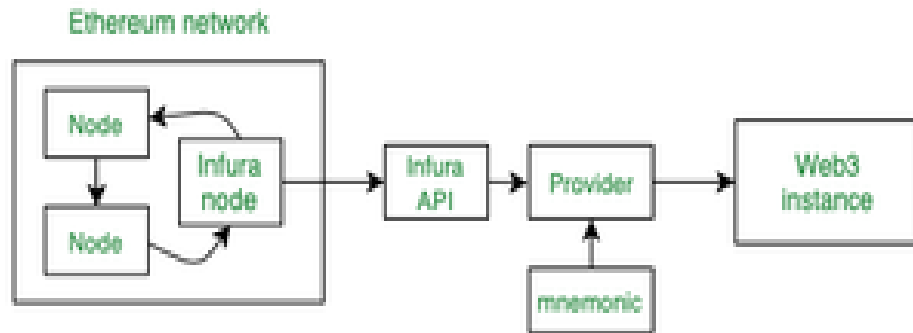
An emulator can be used to deploy a smart contract on a local network eg. Ganache-cli. It takes care of everything and the user doesn't have to worry about the security and the gas amount required for transactions since everything is happening on a local test network. All one has to do is pass the ganache provider as an argument to the web3 instance (web3 facilitates the connection between the blockchain network and the js application).



#### Deploying To Actual Ethereum Network

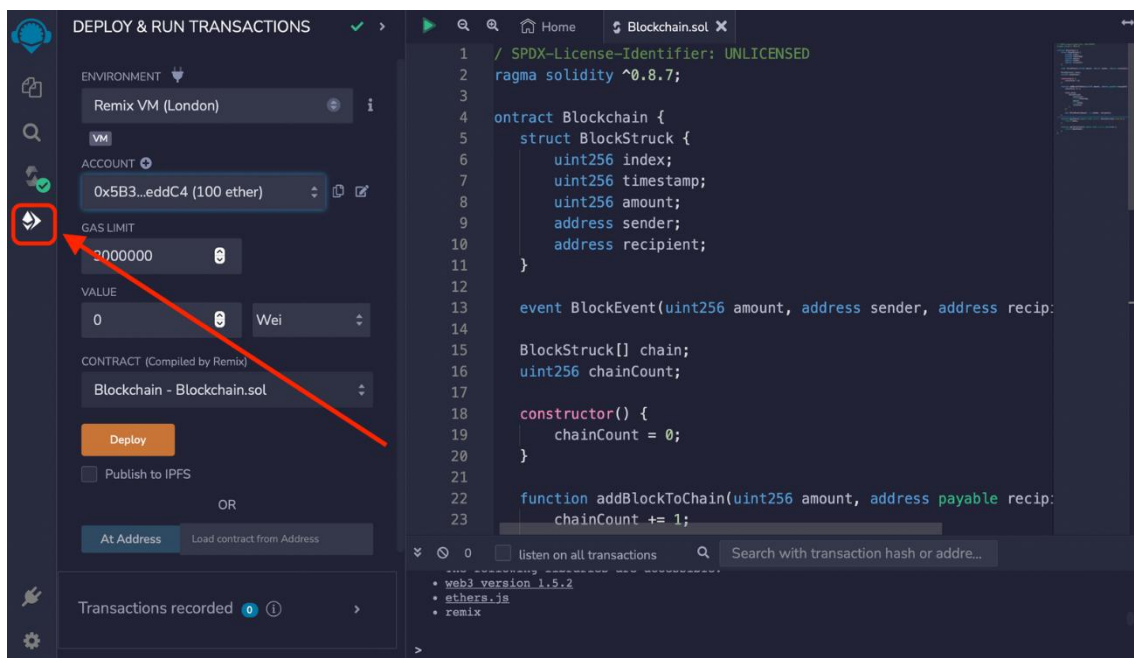
Before deploying a smart contract to an actual Ethereum network make sure the account has some ether in it. Deploying a contract is like sending a transaction and it needs some gas amount to process. Unlike deploying on a local network, transactions will take some time to complete (anywhere between 15 seconds to 5 minutes). Web3 is used to interact with the network the same way it is done in local deployment except customize the provider that will be passed into the web3 instance. Instead of creating our own node that connects to the Ethereum network, one can use a developer platform with RPC endpoints like Infura or Alchemy. With one of these accounts, you have an API key that gives access to their Infura / Alchemy blockchain nodes that are already hosted on the Ethereum network.

Simply sign-up for Infura and get an endpoint that will be used in the code to deploy the smart contract. The below tutorial shows a smart contract being deployed with Infura. For a smart contract tutorial using tools like Alchemy (ethers.js, Hardhat, Solidity, and Metamask), refer to this basic tutorial – “Hello World Smart Contract“.

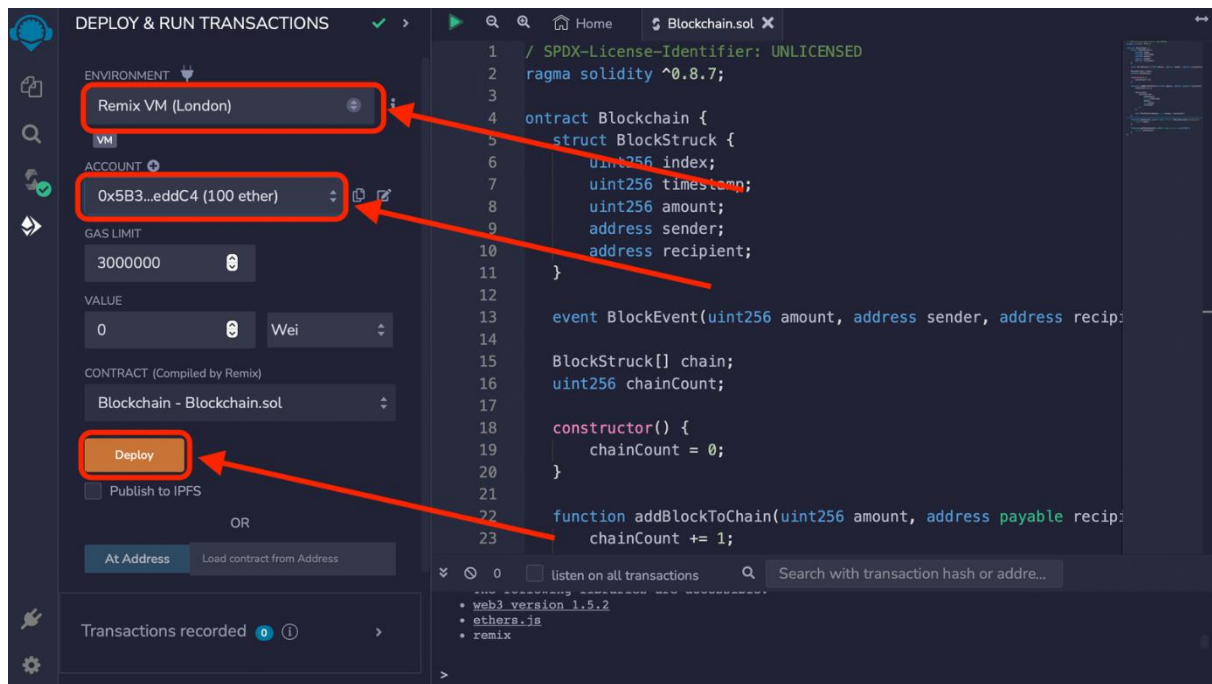


## How to Deploy the Smart Contract

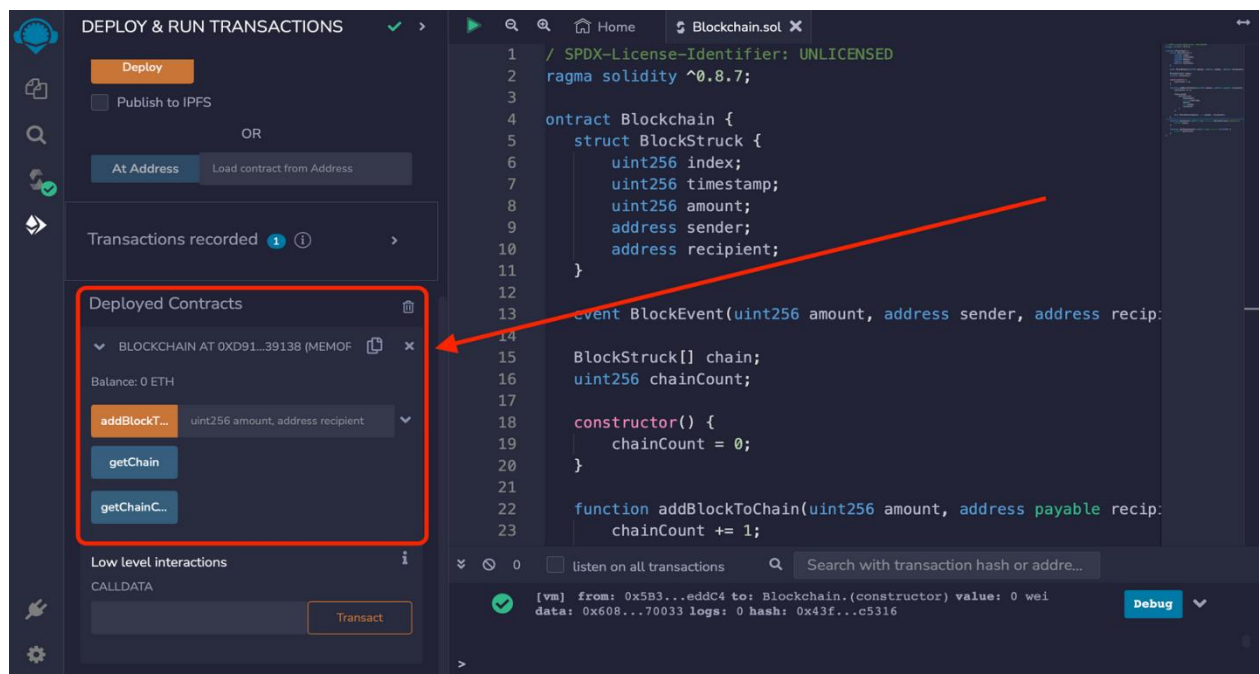
After successful compilation of smart contracts in experiment 2, now we deploy the contract. Click on the fourth icon in the side menu:



Select Remix VM (London) for the ENVIRONMENT. It has ten (10) accounts with 100 dummy ethers each that you may use for test purposes. Then click the Deploy button:



Now when you scroll to the bottom, you will find the Blockchain contract under **Deployed Contracts**. Click the arrow by the deployed contract name to see the functions of the contract that you can interact with.



There are three (3) functions in the image above that match the three (3) functions we defined in our smart contract. Remix automatically creates a UI for you to test your contracts as soon as you deploy them

## How to Test the Smart Contract

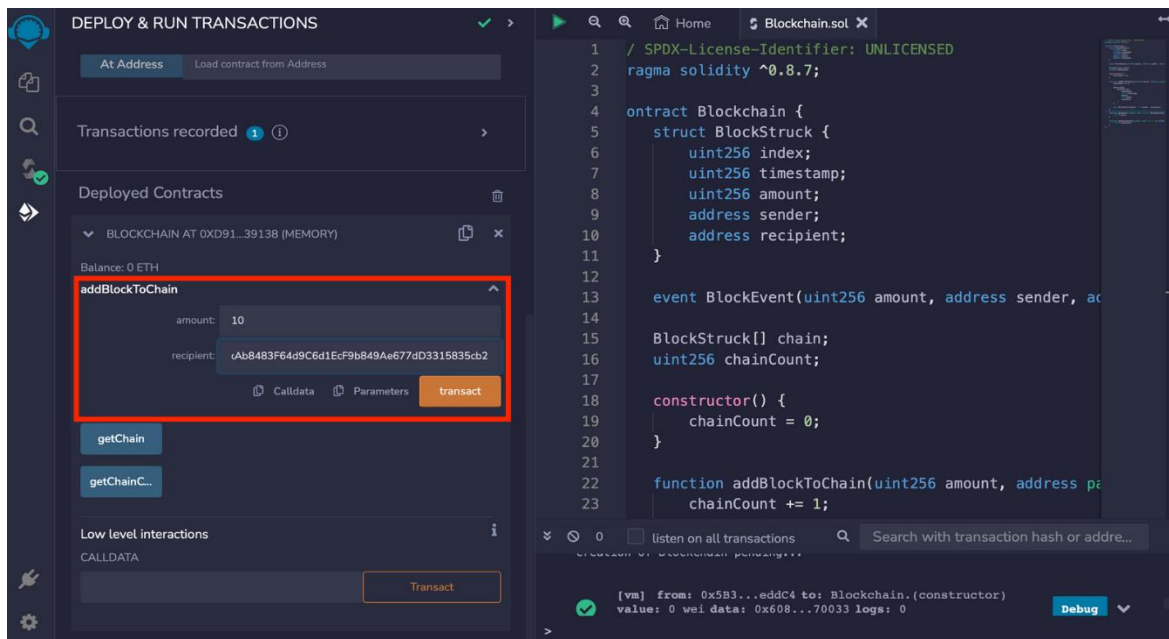
We will now test the functions we created to see how they respond.

### How to test the addBlockToChain function

To test the addBlockToChain function, click the caret (^) icon by the side of the function button and input box. That drops down a form. Fill in 10 for the amount, and fill in one of the ten 10 account addresses for the recipient:

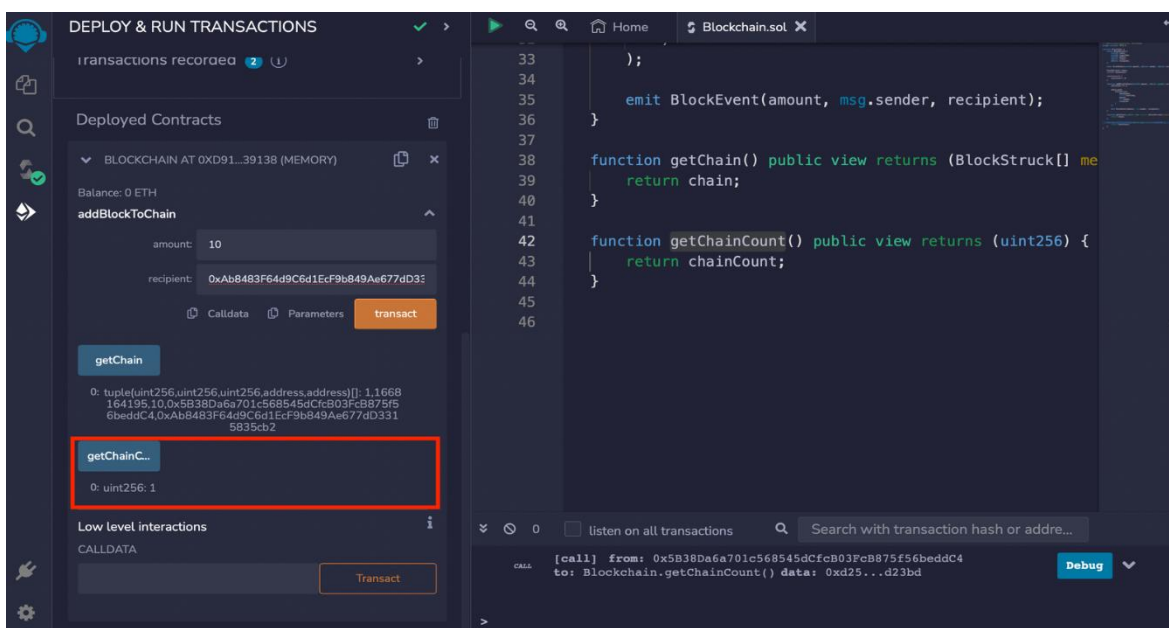
Click the transact button.

Note that you cannot send funds to the same address you used to deploy the contract. You must choose a different account.



### How to test the getChain function

Click the getChain button to reveal the blocks in the chain so far:



And just as we defined it, it returns a uint. There is just one item in the chain for now, but as you keep adding more blocks, the number will increase.

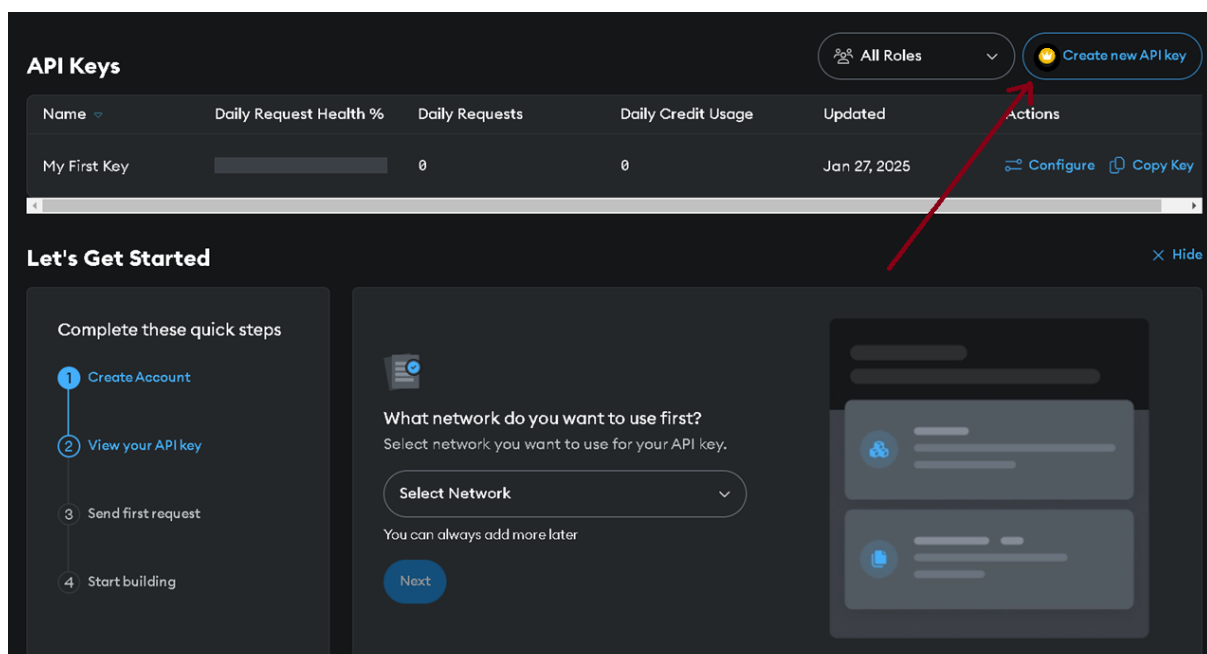
### **Code with steps: [Using Infura]**

```
pragma solidity ^0.8.4;
contract Example
{
    address public manager;
    constructor()
    {
        manager = msg.sender;
    }
}
```

**Step 1:** Install the required dependencies by running the following commands-  
**npm i [solc@0.8.4](#) [truffle-hdwallet-provider@1.0.17](#) [web3@1.3.5](#)**

**Step 2:** Sign up for [Infura](#) and create a project on a particular Ethereum network to get access to the endpoint. The endpoint will be required to deploy the smart contract on the infura node that is already hosted on the Ethereum network. To create a project on infura-

- Click on create a new project.
- Give it a name.
- Select the network to deploy the smart contract on.
- A maximum of 3 projects can be created on infura for free.



**Step 3:** Get access to Bytecode and ABI (Compile the smart contract). Solidity compiler a huge piece of code as output, one can print the output to console if required. Only the relevant part (relevant for deployment) i.e., bytecode and interface are extracted from the output in the following script.

**Compile.js-** Below is the javascript file.

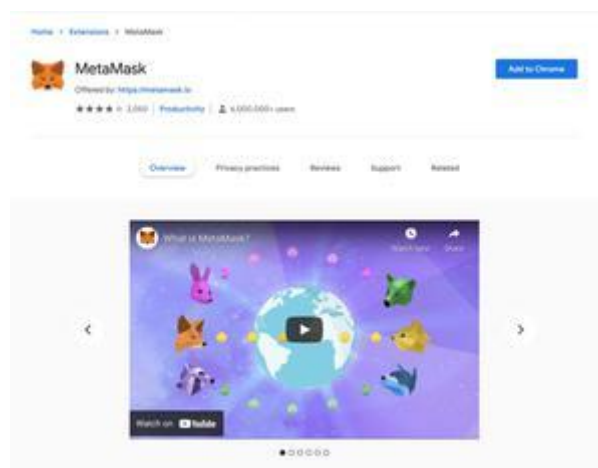
```
const path = require("path");
const fs = require("fs");
const solc = require("solc");

const examplePath = path.resolve(__dirname, "contracts", "example.sol");
const source = fs.readFileSync(examplePath, "utf-8");

var input = {
  language: 'Solidity',
  sources: {
    'example.sol': {
      content: source
    }
  },
  settings: {
    outputSelection: {
      '*': {
        '*': ['*']
      }
    }
  }
};

var output = JSON.parse(solc.compile(JSON.stringify(input)));
var interface = output.contracts["example.sol"]["example"].abi;
var bytecode = output.contracts['example.sol']["example"].evm.bytecode.object;
module.exports = { interface, bytecode };
```

**Step 4:** Add Metamask extension to google chrome from the chrome web store.



**Step 5:** Once have access to the bytecode and interface, all that is required is to create a provider with own mnemonic phrase and infura endpoint using the truffle-hdwallet-provider that was installed earlier.

Create a web3 instance and pass the provider as an argument. Finally, use the deploy method with bytecode as an argument to deploy the smart contract.

### **deploy.js**

```
const HDWalletProvider = require("@truffle/hdwallet-provider");
const Web3 = require("web3");
const { abi, evm } = require("../build/YourContract.json");

const mnemonic = "your twelve word mnemonic here";
const providerURL = "https://sepolia.infura.io/v3/YOUR_INFURA_PROJECT_ID";

const provider = new HDWalletProvider(mnemonic, providerURL);
const web3 = new Web3(provider);

const deploy = async () => {
  try {
    const accounts = await web3.eth.getAccounts();
    console.log("Attempting to deploy from account:", accounts[0]);

    const contract = await new web3.eth.Contract(abi)
      .deploy({ data: evm.bytecode.object })
      .send({ from: accounts[0], gas: "1000000" });

    console.log("Contract deployed at address:", contract.options.address);
    provider.engine.stop();
  } catch (error) {
    console.error("Deployment failed:", error);
  }
};

deploy();
```

### **Output:**

Contract is deployed to 0x8716443863c87ee791C1ee15289e61503Ad4443c

Now the contract is deployed on the network, its functionality can be tested using remix IDE or one can create an interface to interact with the smart contract on the network.

**Conclusion:** Thus, we have successfully developed and publish smart contracts on Ethereum test network.